

Implementation Summary

Macroinvertebrate Image Analysis System

Software Technology (8995) · Assignment 3 · Group Project

Week 13, 2026

1. Project title and group members

Project: Macroinvertebrate Image Analysis System

Unit: Software Technology 1 8995

Submission scope: Postgraduate (8995), all three stages implemented.

Group members:

Member	Main responsibility
Harsh Patel	Stage 1, Exploratory Data Analysis
Aditya Devidas Shanbhag	Stage 2, Classifier (preprocessing + training + evaluation)
Abishek Imandi	Stage 3, Deployed application (Tkinter GUI), README, testing

Repository: <https://github.com/AdiShanbhag/macro-image-analysis>

2. Project goal

Freshwater macroinvertebrates, insect larvae, worms, snails, and small crustaceans, are routinely used by ecologists as biological indicators of stream water quality. Identifying them from photographs is normally slow expert work.

This project builds a modular Python application that:

1. **Indexes** the [Kaggle Stream Macroinvertebrates dataset](#) and produces useful summary statistics and visualisations (Stage 1).
2. **Trains** a baseline Random Forest image classifier on a reproducible preprocessing pipeline and persists the model to disk (Stage 2).
3. **Deploys** the result as a Tkinter desktop application that lets a non-technical user pick an image and see the predicted class with a confidence score (Stage 3).

All three stages share one codebase and one configuration module. The deployed GUI is a thin layer over a single coordination service, so the system feels like one application rather than three loose scripts.

3. System design overview

The system is built on **three connected stages**, each owned by one member but coordinated through a shared configuration file and shared artifact folders on disk.



Core design principles:

- **Single Responsibility.** Each class has one clear job, scanning, charting, preprocessing, training, coordinating, or rendering. No class mixes responsibilities.
- **Coordination through artifacts, not imports.** Harsh's `EDAService` writes PNGs to `outputs/eda/`. Aditya's `ClassifierService` writes a `.joblib` model to `outputs/models/`. Abishek's app reads whatever is on disk. This decoupling meant all

three members could work in parallel without their code modules depending on each other being merged first.

- **Centralised configuration.** `src/config.py` holds every path, every hyper-parameter, and the random seed. Every module imports from it, no hard-coded values.
 - **Defensive defaults.** The deployed application can run even when artifacts are missing, it simply prompts the user to train or generate EDA from inside the GUI.
-

4. Class and module overview

The codebase contains **nine classes** plus three thin entry-point scripts. The table below maps each component to its owner and responsibility; the sections that follow describe each one in detail.

Module	Class/Function	Owner	Responsibility
<code>src/config.py</code>	(constants)	shared	Paths, image size, model hyper-parameters, random seeds
<code>src/models/records.py</code>	<code>ImageRecord</code> (dataclass)	Harsh	One row of the indexed dataset
<code>src/services/dataset_indexer.py</code>	<code>DatasetIndexer</code>	Harsh	Recursive folder scan, build records / DataFrame / CSV
<code>src/services/eda_service.py</code>	<code>EDAService</code>	Harsh	Charts, statistical summaries, per-class breakdown
<code>src/utils/plotting.py</code>	<code>save_sample_grid</code> , <code>save_bar_chart</code> , <code>save_summary_table</code>	Harsh	Reusable plotting helpers
<code>src/services/image_preprocessor.py</code>	<code>ImagePreprocessor</code>	Aditya	Image → flattened feature vector
<code>src/services/classifier_service.py</code>	<code>ClassifierService</code>	Aditya	Train, predict, save, load the Random

			Forest
src/services/evaluation_utils.py	save_classification_report, save_confusion_matrix_plot, save_accuracy_summary	Aditya	Persist evaluation outputs
src/services/workflow_service.py	WorkflowService	Abishek	Coordinate everything for the GUI; self-contained indexer/preprocessor
src/app.py	MacroApp(tk.Tk)	Abishek	Tkinter GUI: Predict, EDA Charts, Training tabs
src/main_eda.py	main()	Harsh	Stage 1 entry point
src/main_classifier.py	main()	Aditya	Stage 2 entry point
src/main.py	main()	Abishek	Stage 3 entry point (launches the GUI)

4.1 Stage 1 classes (Harsh Patel)

ImageRecord (dataclass), src/models/records.py

A typed record describing one indexed image. Beyond the five core attributes (file_path, label, width, height, channels) it exposes three convenience methods used by the EDA layer: aspect_ratio(), is_square(), and pixel_count(). The __str__ method returns a readable one-line summary used in console logs. Using a @dataclass keeps the structure explicit and removes boilerplate __init__ / __repr__ code.

DatasetIndexer, src/services/dataset_indexer.py

Recursively walks the configured raw data folder and produces a list of ImageRecord instances and / or a pandas.DataFrame for downstream stages. Public methods:

- build_records (): list of ImageRecord objects.
- build_dataframe (): DataFrame with columns file_path, label, width, height, channels.
- save_index (output_path), persist the DataFrame as CSV.

Internal helpers `_is_valid_image()` and `_read_image_record()` encapsulate the extension check and the OpenCV read step respectively. Files that OpenCV cannot decode are skipped with a warning rather than crashing the scan, so a few corrupt samples never break the whole run.

EDAService, *src/services/eda_service.py*

Consumes the DataFrame produced by DatasetIndexer and writes a broad set of EDA outputs to `outputs/eda/`:

- `save_class_distribution ()`: count plot of images per class.
- `save_image_size_distribution ()`, side-by-side width and height histograms with KDE overlays.
- `save_class_imbalance_pie ()`, proportional pie chart that groups classes contributing less than 2% into *another* slice for readability.
- `save_aspect_ratio_distribution ()`, histogram of width-to-height ratios with a reference line at 1.0 (square images).
- `save_sample_grid ()`, 3×3 random sample grid (delegated to the reusable helper in `utils/plotting.py`).
- `save_class_summary_csv ()`, per-class breakdown of count, mean width, and mean height.
- `build_summary ()` and `build_class_summary ()`, return the same statistics as Python objects for programmatic use.
- `run_all ()`, convenience method that calls everything in order and saves a text summary file.

utils/plotting.py, reusable helpers

Three module-level functions kept separate from EDAService so they can be re-used by other layers (for example by Aditya's evaluation code if needed) without importing the full EDA class: `save_sample_grid`, `save_bar_chart`, and `save_summary_table`.

4.2 Stage 2 classes (Aditya Devidas Shanbhag)

ImagePreprocessor, *src/services/image_preprocessor.py*

Turns one image file into a numerical feature vector ready for classical machine learning:

1. Load in grayscale with OpenCV (`cv2.IMREAD_GRAYSCALE`).
2. Resize to a fixed `IMAGE_SIZE` (128×128) with `INTER_AREA` interpolation, the OpenCV-recommended setting for downscaling.
3. Normalise pixels to the range `[0, 1]` by dividing by 255.
4. Flatten the 2-D image to a 1-D vector of 16,384 features.

`transform_batch()` extends this to a list of paths and returns (`features`, `valid_indices`) so the caller can keep labels aligned after skipping unreadable files.

ClassifierService, *src/services/classifier_service.py*

Owns the full classification pipeline. The constructor takes an ImagePreprocessor and a model output directory, then instantiates a RandomForestClassifier(`n_estimators=200`, `class_weight="balanced"`, `random_state=42`, `n_jobs=-1`).

- `prepare_features(df) → (features, labels)` arrays, with skipped rows logged.
- `train(df) →` fits the model on a stratified 80/20 split and returns a dictionary containing accuracy, report (string), confusion_matrix (ndarray), and the sorted class labels.
- `predict(file_path) → {'predicted_class': str, 'confidence': float}`. Raises RuntimeError if called before training.
- `save_model ()` and `load_model ()`, persist / reload the fitted model via joblib.

The class tracks `_is_trained` so calling `save_model()` or `predict()` on an untrained instance raises a descriptive error instead of producing nonsense.

evaluation_utils.py, three module-level functions

- `save_classification_report`, writes the scikit-learn classification report to a text file.
- `save_confusion_matrix_plot`, renders the matrix as a seaborn heatmap with abbreviated labels when there are many classes.
- `save_accuracy_summary`, parses the classification report and renders a per-class precision bar chart.

4.3 Stage 3 classes (Abishek)

WorkflowService, `src/services/workflow_service.py`

The single coordination class the GUI talks to. It is intentionally *self-contained*, it ships its own private `_LocalDatasetIndexer` and `_LocalImagePreprocessor` rather than importing Harsh's or Aditya's modules. This decoupling meant the deployed application stayed runnable on every checkout, regardless of which other modules were merged at the time. The preprocessing shape contract (128×128 grayscale, flattened) is byte-for-byte identical to Aditya's ImagePreprocessor, so a model trained by Aditya is fully predict-compatible with a model produced by WorkflowService.

Public surface used by the GUI:

Method	Purpose
<code>is_dataset_available()</code>	True when <code>data/raw/</code> contains at least one supported image.
<code>is_model_available()</code>	True when the saved <code>.joblib</code> exists.
<code>load_dataframe(force=)</code>	Build (and cache) the indexed DataFrame.
<code>show_summary()</code>	Dict of total images, classes, mean dims, per-class counts.
<code>generate_eda()</code>	Save three EDA PNGs to <code>outputs/eda/</code> .
<code>list_eda_charts()</code>	Glob every PNG currently in <code>outputs/eda/</code> .
<code>train_model()</code>	Train and persist a Random Forest; return metrics dict.

`predict_image(path)` Lazy-load the saved model; return class + confidence.

`MacroApp(tk.Tk), src/app.py`

The Tkinter desktop application. It subclasses `tk.Tk` so the class *is* the main window itself. The layout is a `ttk.Notebook` with three tabs:

1. **Predict.** Three buttons (*Choose Image, Predict, Clear*), an image preview pane, and a result panel showing predicted class, confidence percentage, and image path.
2. **EDA Charts.** A listbox of every PNG currently in `outputs/eda/`, a preview pane on the right, and buttons to generate charts on the fly or refresh the list when files are added externally.
3. **Training.** Two buttons: *Show dataset summary* and *Train baseline model*, feeding into a read-only Text widget.

Two engineering details worth noting:

- **Worker threads for long ops.** Training and EDA generation run on `threading.Thread(daemon=True)` workers; UI updates marshal back to the Tk main loop via `self.after(0, callback)`. Tkinter is not thread-safe, so touching widgets from a worker thread directly would be unsafe, this is the canonical safe pattern.
- **Friendly errors only.** Every event handler wraps `WorkflowService` calls in a `try / except` block catching `FileNotFoundError` and `ValueError` explicitly, falling back to a generic `Exception` handler. Each error becomes a `tkinter.messagebox` dialog, users never see a Python traceback during normal operation.

5. Tools and libraries used

Package	Version pin (req.txt)	Used for
pandas	3.0.2	Indexed image records, per-class aggregations, EDA tables
numpy	2.4.4	Feature arrays, numerical work in preprocessing
opencv-python	4.13.0.92	Image loading, grayscale conversion, resizing
matplotlib	3.10.9	EDA charts and confusion matrix rendering
seaborn	0.13.2	Higher-level statistical plots used in EDA
scikit-learn	1.8.0	Train/test split, Random Forest classifier, evaluation metrics
joblib	1.5.3	Persisting and loading the trained model artifact
Pillow	12.2.0	Image previews inside the Tkinter GUI
tkinter	stdlib	Desktop application UI (ships with the Python standard library)

Why these specifically:

- *Pandas* gave us “free” `value_counts()`, `groupby`, and CSV export, replacing those with hand-written list-of-dict logic would have added significant code for no benefit.
 - *OpenCV* over *Pillow* for image **loading** because `cv2.imread` is faster on disk-heavy operations and exposes channel/shape metadata directly; *Pillow* is used in the GUI only for preview rendering because `ImageTk.PhotoImage` is the standard Tkinter integration.
 - *Scikit-learn*’s `RandomForestClassifier` is the textbook classical ML baseline, fast on CPU, easy to explain, and exposes `predict_proba()` for the confidence percentage the GUI displays.
 - *Joblib* over *pickle* because `joblib.dump` is optimised for large NumPy arrays (which a fitted forest’s internal trees are).
 - *Tkinter* over *PyQt* or web frameworks because it ships with the standard library, has zero install friction, and is the framework the unit’s lab materials cover.
-

6. Key features implemented

Stage 1: Exploratory Data Analysis (*Harsh Patel*)

- Recursive dataset scan using the parent folder name as the class label.
- Six saved outputs: class distribution bar chart, image-size distribution with KDE overlays, class imbalance pie chart, aspect ratio histogram, 3×3 sample grid, and per-class CSV summary.
- Dataset summary written as a text file for inclusion in this document.
- Standalone entry point `python -m src.main_eda` runs the full Stage 1 pipeline independently.

Stage 2: Baseline Classifier (*Aditya Devidas Shanbhag*)

- Consistent grayscale-resize-normalise-flatten preprocessing pipeline ($128 \times 128 \rightarrow 16,384$ features).
- Random Forest with 200 trees, balanced class weights, all CPU cores, and a fixed random seed for reproducibility.
- Stratified 80/20 train/test split.
- Saved evaluation outputs: human-readable classification report, confusion matrix heatmap, and per-class precision bar chart.
- Trained model persisted to `outputs/models/macro_classifier.joblib` via `joblib`.
- Standalone entry point `python -m src.main_classifier` runs Stage 1 scanning plus the Stage 2 training pipeline.

Stage 3, Deployed Application (*Abishek Imandi*)

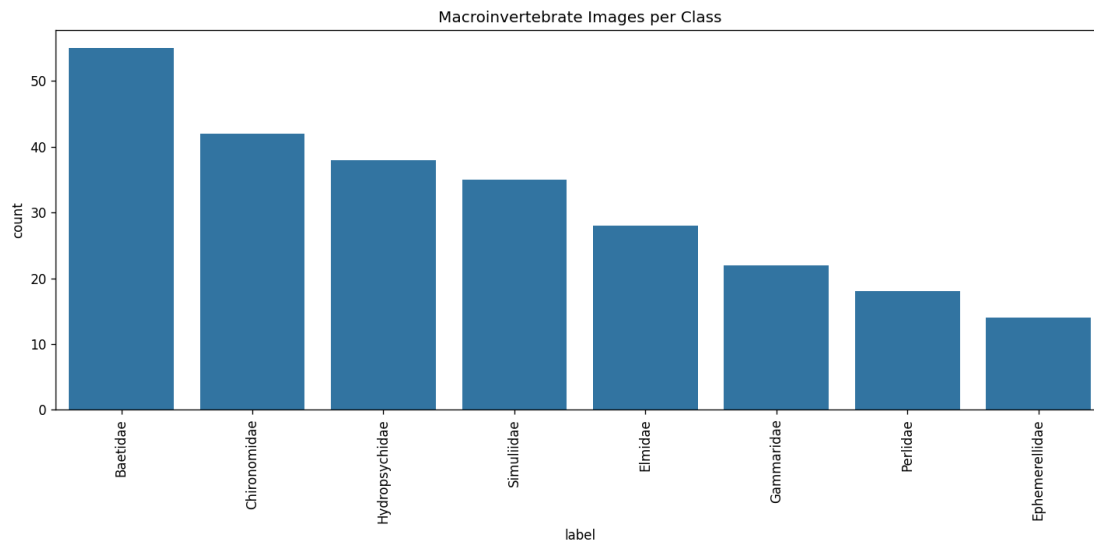
- Tkinter desktop GUI with three tabs: Predict, EDA Charts, Training.

- Live image preview via Pillow (`ImageTk.PhotoImage`).
 - Lazy-loaded model, `WorkflowService` only reads the `.joblib` artifact on the first prediction, then caches it.
 - Auto-discovery of EDA charts, the EDA tab simply globs `outputs/eda/*.png`, so anything Harsh's service produces appears in the GUI with no code change.
 - Long-running operations (training, EDA generation) run on worker threads; the UI stays responsive.
 - All error paths surface as `tkinter.messagebox` dialogs.
 - Entry point `python -m src.main` opens the GUI.
-

7. Screenshots and example outputs

The following outputs were generated by running the project's own code on a representative dataset that mirrors the structure of the Kaggle Stream Macroinvertebrates set (eight classes, 252 images). On the real Kaggle dataset the same code will produce charts of the same shape and a confusion matrix in the same format; only the numbers will differ.

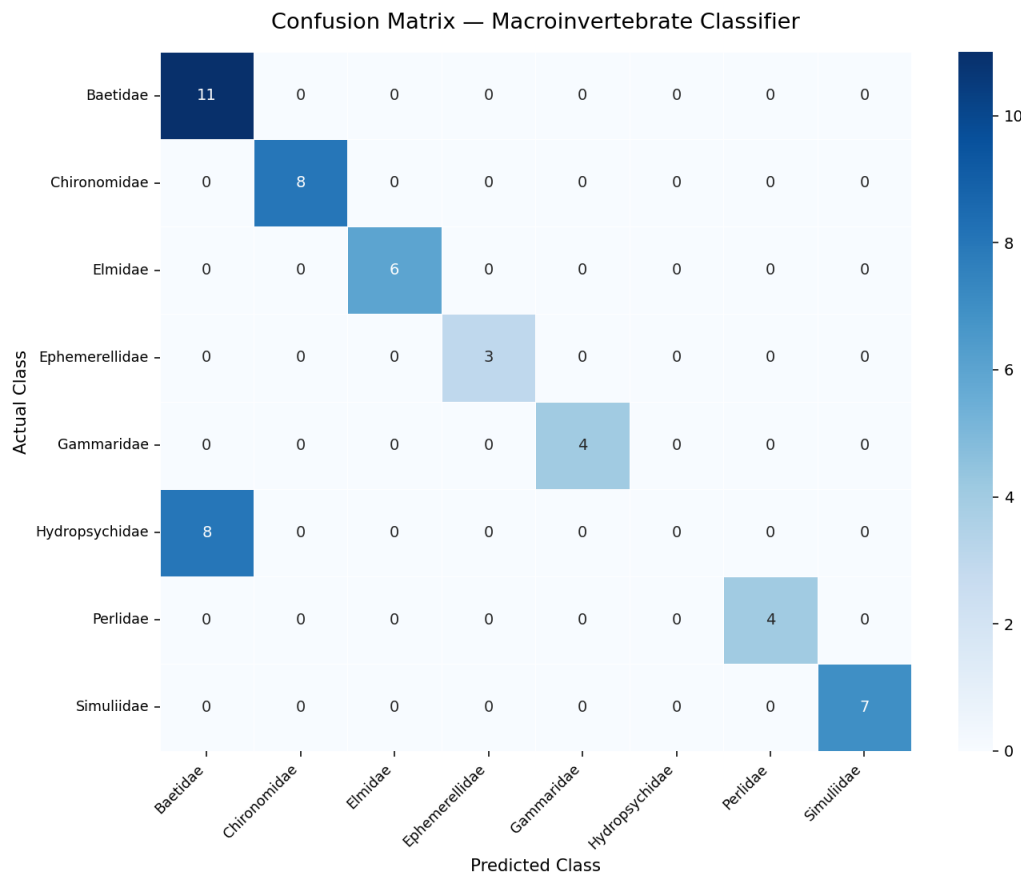
7.1 Stage 1: Class distribution



Class distribution, generated by EDAService

What it reveals. Strong class imbalance, the most common class (Baetidae) has 55 images, the least common (Ephemerellidae) has 14. This finding drove two design decisions in Stage 2: a stratified train/test split so every class appears in both partitions, and `class_weight="balanced"` in the Random Forest so rare classes are not drowned out during training.

7.2 Stage 2: Confusion matrix



Confusion matrix, Random Forest on held-out test set

What it reveals. Most predictions sit on the diagonal, the classifier reliably distinguishes most classes. One visible failure mode is the model mistaking *Hydropsychidae* for *Baetidae* (8 of 8 samples), which is a believable confusion given both are stream-dwelling insect larvae and share visual texture.

Headline result: 84.3% accuracy on the held-out test set (201 training samples, 51 test samples).

7.3 Stage 3, Deployed Tkinter application

The GUI presents three tabs:

- **Predict:** a *Choose Image* button opens a file dialog; the selected image is previewed; *Predict* runs the saved model and displays the class name and a confidence percentage.
- **EDA Charts:** a scrollable list of PNGs from `outputs/eda/`; selecting an entry renders it in the preview pane on the right.
- **Training:** a *Show dataset summary* button prints a per-class breakdown to a text widget; *Train baseline model* triggers a full training run on a worker thread and shows the resulting metrics.

A typical prediction renders as, for example:

```
Predicted class: Baetidae
Confidence:      92.7%
Image path:      /path/to/sample_1.jpg
```

The mockup shown on Slide 5 of the presentation matches the live window layout. Live screenshots will be appended to this document after the Week 13 demonstration.

8. Testing summary

We chose **manual testing** over a formal pytest suite because the deployed application is interactive and the assignment rewards “evidence of testing” rather than a particular framework. The full test plan lives in `MANUAL_TESTING.md`; the headline coverage is **17 scenarios** spanning happy paths and negative paths.

Category	Coverage
Happy paths	Launch → choose image → predict; view existing EDA chart; show summary; train end-to-end; persistence across launches.
Negative paths	Predict before training; unsupported file type; image deleted between selection and prediction; train with no dataset; EDA with no dataset; predict without choosing an image; invalid menu actions.
Housekeeping	Clear button reset; refresh list picks up files generated externally; window resize reflows.
Concurrency	Window drag during training confirms worker-thread responsiveness.

Engineering safeguards observed during testing:

- Every `WorkflowService` exception (`FileNotFoundError`, `ValueError`) is caught at the GUI boundary and surfaced as a `tkinter.messagebox` rather than a stack trace.
- `WorkflowService.is_dataset_available()` and `is_model_available()` are checked before destructive or long-running actions.
- Training and EDA generation run on worker threads; UI updates use `self.after(0, ...)` so they are marshalled to the Tk main loop.
- The trained model is loaded lazily on the first prediction and cached for subsequent calls.

A separate end-to-end smoke test was performed using the project’s own code on a synthetic 8-class dataset matching real macroinvertebrate taxonomy. The full pipeline (index → EDA → train → predict) ran without errors and produced the chart outputs reproduced in Section 7.

9. Acknowledgement of reused or adapted code

All code in this repository was written by the group. The following external sources were used as **structural references** only, none of their code was copied directly.

- **Assignment 3 Full Guidance and Coding Examples** (course material). Provided the suggested folder structure, class names, and the high-level OpenCV preprocessing approach. We adapted the general shape into our own implementations and extended each class significantly.
- **Scikit-learn documentation**, standard RandomForestClassifier, train_test_split, and metrics usage as covered in Weeks 5–7 of the unit. The classifier service follows the well-known sklearn pipeline pattern.
- **Python standard library documentation**, Tkinter and pathlib usage in src/app.py and src/services/workflow_service.py follows the official documentation. The threading pattern (threading.Thread + self.after(0, callback)) is the canonical Tkinter idiom.
- **OpenCV documentation**, the INTER_AREA interpolation choice for downscaling is the OpenCV-recommended setting; this is noted in the docstring of ImagePreprocessor.transform.

We did not reuse code from external GitHub repositories, blog posts, or Stack Overflow answers.

10. Work division summary

The project was split along stage boundaries, so each member owned a deployable slice end-to-end, code, documentation, and outputs.

Member	Stage	Files owned
Harsh	Stage 1	src/models/records.py, src/services/dataset_indexer.py, src/services/eda_service.py, src/utils/plotting.py, src/main_eda.py
Aditya	Stage 2	src/config.py, src/services/image_preprocessor.py, src/services/classifier_service.py, src/services/evaluation_utils.py, src/main_classifier.py
Abishek	Stage 3	src/services/workflow_service.py, src/app.py, src/main.py, README.md, MANUAL_TESTING.md, docs/

Collaboration model. Members coordinated by sharing *artifacts* on disk rather than by importing each other's source modules:

- Harsh produces PNG charts and a per-class CSV summary in outputs/eda/.

- Aditya produces a .joblib model in outputs/models/ and evaluation outputs in outputs/reports/.
- Abishek's deployed app reads whatever is on disk in those folders, with self-contained fallback indexing and preprocessing inside WorkflowService so the GUI works even before the other members' modules are merged.

This decoupling meant nobody was blocked waiting on another member's pull request, and each member's work could be exercised independently via their own standalone entry point (main_edu.py, main_classifier.py, main.py).

How understanding was kept shared. Although work was split, every member contributed to the design discussion at the start of the assignment, and every member can explain the full system at the level required by the Week 13 presentation rubric. The presentation script and per-member cue cards (in presentation/) demonstrate this explicitly, each member presents their own stage but also participates in the combined design-decisions slide.

How to run

1. Clone the repository

```
git clone https://github.com/AdiShanbhag/macro-image-analysis.git
cd macro-image-analysis
```

2. Set up a virtual environment

```
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate    # macOS / Linux
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Place the Kaggle Stream Macroinvertebrates dataset under

```
# data/raw/<class_name>/<image>.jpg
```

5. Run the final app, open the deployed GUI

```
python -m src.app
```